

.NET Senior

Interview Questions & Answers

50 Real-World Questions with Clear Answers

C# & .NET | ASP.NET Core | EF Core | Microservices | Security | System Design
Docker | Kubernetes | CI/CD | gRPC | GraphQL | PostgreSQL | Redis

2026 Edition

■ .NET & ASP.NET Core

Q1 Explain the complete request processing pipeline in ASP.NET Core.

Answer

When a request arrives, Kestrel (the HTTP server) receives it and passes it through a middleware pipeline. Each middleware can read, modify, or short-circuit the request before passing it to the next one. The response travels back through the same pipeline in reverse.

Correct order: Kestrel → UseExceptionHandler → UseRouting → UseAuthentication → UseAuthorization → MapControllers (endpoint) → your action method → response returns in reverse.

Think of middleware like airport security checkpoints — every passenger (request) goes through each gate in order.

Example

```
app.UseExceptionHandler("/error"); // catches unhandled exceptions
app.UseRouting();                  // matches URL to endpoint
app.UseAuthentication();           // who are you?
app.UseAuthorization();            // what can you do?
app.MapControllers();              // dispatch to controller
```

Q2 How does Dependency Injection work internally in ASP.NET Core?

Answer

ASP.NET Core has a built-in IoC container. You register services at startup, and the framework injects them wherever they are needed — constructor, action method, or middleware.

Internally the container builds an object graph: it looks at the constructor, resolves each parameter from its registry, and hands you a fully wired object.

Example

```
// Registration
builder.Services.AddScoped<IOrderService, OrderService>();

// Usage - framework injects it automatically
public class OrderController(IOrderService svc) : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult Get(int id) => Ok(svc.GetOrder(id));
}
```

Q3 What are the differences between Scoped, Singleton, and Transient services?

Answer

Transient — new instance every single time it is requested. Good for lightweight, stateless helpers. Scoped — one instance per HTTP request. Perfect for DbContext, unit-of-work patterns. Singleton — one instance for the entire application lifetime. Use for caches, config readers, thread-safe services.

Danger: injecting a Scoped service into a Singleton causes a "captive dependency" bug — the Scoped instance never gets released.

Example

```
services.AddTransient<IEmailSender, SmtEmailSender>(); // new each time
services.AddScoped<AppDbContext>();                  // per request
services.AddSingleton<IMemoryCache, MemoryCache>();   // app lifetime
```

Q4 How do you identify and resolve memory leaks in .NET applications?

Answer

Common culprits: static event handlers that keep objects alive, undisposed HttpClient/DbContext, large object heap fragmentation, and background services holding references.

Diagnosis: dotnet-counters, dotnet-dump, Visual Studio Diagnostic Tools, or JetBrains dotMemory. Look for Gen2 GC collections growing over time.

Fix pattern: always implement IDisposable, use 'using' statements, prefer IHttpClientFactory over raw HttpClient.

Example

```
// Wrong - leaks a socket every call
var client = new HttpClient();

// Right - reuses pooled handlers
public class MyService(IHttpClientFactory factory)
{
    public async Task<string> GetData()
    {
        using var client = factory.CreateClient("api");
        return await client.GetStringAsync("/endpoint");
    }
}
```

Q5

What performance optimization techniques have you implemented in .NET applications?

Answer

Key techniques: async/await everywhere to free thread-pool threads, response caching, output caching, Span/Memory to avoid allocations, compiled EF Core queries, connection pooling, and using ArrayPool instead of allocating new arrays in hot paths.

Example

```
// Avoid allocations in hot path with Span
public static int SumDigits(ReadOnlySpan<char> input)
{
    int total = 0;
    foreach (char c in input)
        if (char.IsDigit(c)) total += c - '0';
    return total;
}

// Compiled EF query (parsed once, reused many times)
private static readonly Func<AppDbContext, int, Task<Order?>> GetOrderById =
    EF.CompileAsyncQuery((AppDbContext db, int id) =>
        db.Orders.FirstOrDefault(o => o.Id == id));
```

Q6

How do you handle application configuration across multiple environments?

Answer

Use layered configuration: appsettings.json as base, appsettings.{Environment}.json for overrides, environment variables for secrets in production, and Azure Key Vault / AWS Secrets Manager for sensitive values.

Never commit secrets to source control. Use the Options pattern to bind config to typed classes.

Example

```
// appsettings.Development.json
{ "ConnectionStrings": { "Db": "Server=localhost;..." } }

// Bind to typed options
builder.Services.Configure<DatabaseOptions>(
    builder.Configuration.GetSection("ConnectionStrings"));

// Inject and use
public class Repo(IOptions<DatabaseOptions> opts)
{
    private readonly string _conn = opts.Value.Db;
}
```

Q7 What are common causes of high CPU and memory usage in .NET applications?

Answer

CPU spikes: tight loops without await, excessive LINQ materialisation, regex without compilation, synchronous I/O on thread-pool threads (blocking calls). Memory: large object heap pressure, string concatenation in loops, event handler leaks, loading entire datasets into RAM. Quick wins: profile with dotnet-trace, add async/await, use IEnumerable for streaming, paginate database queries.

Example

```
// Bad - loads ALL orders into RAM
var orders = await db.Orders.ToListAsync();

// Good - stream one row at a time, no RAM spike
await foreach (var order in db.Orders
    .AsNoTracking()
    .AsAsyncEnumerable())
{
    await ProcessAsync(order);
}
```

Q8 How would you implement caching in a high-traffic application?

Answer

Layer your caches: in-process IMemoryCache for tiny hot data (microsecond reads), IDistributedCache backed by Redis for shared state across pods, and HTTP response caching / CDN for static or semi-static API responses.

Apply Cache-Aside pattern: try cache → on miss, load from DB → store in cache → return.

Example

```
public async Task<Product?> GetProduct(int id)
{
    string key = $"product:{id}";
    if (_cache.TryGetValue(key, out Product? cached)) return cached;

    var product = await _db.Products.FindAsync(id);
    if (product is not null)
        _cache.Set(key, product, TimeSpan.FromMinutes(10));
    return product;
}
```

■ Entity Framework Core & Database

Q9 How do you optimize slow EF Core queries?

Answer

First: turn on query logging and find what SQL gets generated. Common fixes — add .AsNoTracking() for read-only queries, select only needed columns with .Select(), add proper indexes, use compiled queries for hot paths, and avoid loading navigation properties you don't need (disable lazy loading in perf-sensitive code).

Example

```
// Slow - loads full entity + change tracking overhead
var allOrders = await db.Orders.ToListAsync();

// Fast - only needed columns, no change tracking
var pendingOrders = await db.Orders
    .AsNoTracking()
    .Where(o => o.Status == "Pending")
    .Select(o => new { o.Id, o.Total })
    .ToListAsync();
```

Q10 Explain the N+1 Query Problem and its solutions.

Answer

N+1 happens when you load a list (1 query) then access a navigation property inside a loop, triggering 1 extra query per row = N+1 total. Solutions: eager loading with `.Include()`, projection with `.Select()`, or split queries (`.AsSplitQuery()`).

Example

```
// Bad - N+1: 1 query for orders + 1 extra query per row for customer
var orders = await db.Orders.ToListAsync();
foreach (var o in orders)
    Console.WriteLine(o.Customer.Name); // triggers a DB call per row!

// Good - single JOIN, all data in one round trip
var ordersWithCustomer = await db.Orders
    .Include(o => o.Customer)
    .AsNoTracking()
    .ToListAsync();
```

Q11 When would you choose Dapper over Entity Framework Core?

Answer

Use Dapper when: you need maximum query performance, you're writing complex raw SQL that EF can't express cleanly, you're working with stored procedures or dynamic SQL, or the schema doesn't map well to entities.

Use EF Core when: you want migrations, change tracking, LINQ queries, and faster CRUD development.

Example

```
// Dapper - raw SQL, fast, explicit
var dapperOrders = await conn.QueryAsync<Order>(
    "SELECT Id, Total FROM Orders WHERE CustomerId = @id",
    new { id = customerId });

// EF Core - LINQ, migrations, change tracking
var efOrders = await db.Orders
    .Where(o => o.CustomerId == customerId)
    .ToListAsync();
```

Q12 How do you troubleshoot a poorly performing query in production?

Answer

Step 1: capture the actual SQL (EF logging, SQL Server Profiler, `pg_stat_statements` for Postgres). Step 2: run `EXPLAIN ANALYZE` in Postgres or Execution Plan in SSMS. Step 3: look for Sequential Scans / Table Scans — add indexes. Step 4: check for parameter sniffing, outdated statistics, or missing covering indexes. Step 5: test fix in staging before deploying.

Example

```
-- Postgres: see what the query planner does
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 42 AND status = 'Pending';
-- Look for: Seq Scan (bad) vs Index Scan (good)
-- Fix: CREATE INDEX idx_orders_customer_status
--      ON orders(customer_id, status);
```

Q13 What indexing strategies have you implemented in large databases?

Answer

Composite indexes for multi-column WHERE/ORDER BY, covering indexes (INCLUDE columns) to avoid table lookups, partial indexes for filtered subsets (e.g. WHERE deleted = false), and avoiding over-indexing which slows writes.

Rule: index columns you filter/sort on frequently, in the order they appear in WHERE clauses.

Example

```
-- Composite index: filters by tenant then date
CREATE INDEX idx_orders_tenant_date
ON orders(tenant_id, created_at DESC);

-- Covering index: query fulfilled from index alone
CREATE INDEX idx_orders_covering
ON orders(customer_id)
INCLUDE (total, status);

-- Partial index: only index active records
CREATE INDEX idx_users_active
ON users(email) WHERE deleted_at IS NULL;
```

Q14 How do you handle database migrations in production environments?

Answer

Never run dotnet ef database update in production from a dev machine. Instead: generate migration SQL scripts (--idempotent), review them, test in staging, then apply via CI/CD pipeline or a migration tool (Flyway, Liquibase).

Use a migrations history table so you can track what ran. Always have a rollback script ready.

Example

```
# Generate idempotent SQL script
dotnet ef migrations script --idempotent -o migration.sql

# In CI/CD pipeline (e.g. GitHub Actions)
- name: Apply DB migrations
  run: |
    psql ${ secrets.DB_URL } -f migration.sql

# EF Core bundle approach (single executable)
dotnet ef migrations bundle --self-contained -o migrate
```

Q15 What are the differences between Clustered and Non-Clustered Indexes?

Answer

Clustered index — the table data IS physically sorted by this index. Only one per table. In SQL Server the primary key is clustered by default. Range queries on the clustered key are very fast.

Non-Clustered index — a separate structure pointing back to the data row. You can have many. Used for secondary lookup columns.

Example

```
-- SQL Server: PK creates clustered index automatically
CREATE TABLE Orders (
    Id INT PRIMARY KEY,          -- clustered
    CustomerId INT,
    CreatedAt DATETIME
);

-- Non-clustered on a frequently filtered column
CREATE NONCLUSTERED INDEX IX_Orders_Customer
ON Orders(CustomerId) INCLUDE (CreatedAt);

-- PostgreSQL: explicit clustered-like behaviour
CLUSTER orders USING orders_pkey;
```

Q16 How do you improve SQL query performance for large datasets?

Answer

Key techniques: proper indexes, avoid SELECT *, use pagination (OFFSET/FETCH or keyset), partition large tables, rewrite correlated sub-queries as JOINS, use CTEs for readability + let the optimiser work, and archive old data.

Example

```
-- Bad: loads all rows then filters in app
SELECT * FROM orders;

-- Good: filter + paginate at DB level
SELECT id, total, created_at
FROM orders
WHERE tenant_id = @tid AND status = 'Completed'
ORDER BY created_at DESC
LIMIT 50 OFFSET 0;    -- Postgres keyset is even faster:
-- WHERE created_at < @lastSeen ORDER BY created_at DESC LIMIT 50
```

■ Microservices & Distributed Systems

Q17 How would you design a Microservice Architecture from scratch?

Answer

Start with domain decomposition (DDD Bounded Contexts) — each service owns its data and exposes an API. Key decisions: synchronous REST/gRPC for queries, async messaging (Kafka/Service Bus) for events, an API Gateway for client-facing routing, and a service mesh (Istio/Linkerd) for observability.

Example service boundaries: OrderService, InventoryService, PaymentService, NotificationService — each with its own DB.

Example

```
// Each service has its own DbContext / schema
// OrderService
[HttpPost("orders")]
public async Task<IActionResult> PlaceOrder(OrderRequest req)
{
    var order = await _orderSvc.Create(req);
    // Publish event - don't call PaymentService directly
    await _bus.PublishAsync(new OrderPlacedEvent(order.Id, order.Total));
    return CreatedAtAction(nameof(Get), new { id = order.Id }, order);
}
```

Q18 What challenges have you faced implementing Microservices?

Answer

Top real-world challenges: 1. Distributed transactions — no shared DB, so use Saga pattern. 2. Data consistency — eventual consistency requires careful design. 3. Network failures — add retries, circuit breakers. 4. Service discovery — use Kubernetes DNS or Consul. 5. Observability — correlate logs/traces across services with a correlation ID. 6. Local development complexity — Docker Compose helps.

Example

```
// Correlation ID middleware - trace a request across all services
app.Use(async (ctx, next) =>
{
    var correlationId = ctx.Request.Headers["X-Correlation-Id"]
        .FirstOrDefault() ?? Guid.NewGuid().ToString();
    ctx.Response.Headers["X-Correlation-Id"] = correlationId;
    using var scope = logger.BeginScope(
        new Dictionary<string, object> { ["CorrelationId"] = correlationId });
    await next();
});
```

Q19 How do you manage communication between Microservices?

Answer

Synchronous: REST (simple, widely understood) or gRPC (fast, typed contracts, great for internal service-to-service). Asynchronous: message broker (RabbitMQ, Kafka, Azure Service Bus) for events and commands that don't need an immediate response.

Rule: prefer async for anything that can tolerate eventual consistency. Use sync only when you need an immediate answer.

Example

```
// gRPC service definition (proto)
service InventoryService {
    rpc CheckStock (StockRequest) returns (StockResponse);
}

// C# client call
var channel = GrpcChannel.ForAddress("https://inventory-svc");
var client = new Inventory.InventoryClient(channel);
var reply = await client.CheckStockAsync(
    new StockRequest { ProductId = productId, Quantity = 5 });
if (!reply.Available) return BadRequest("Out of stock");
```

Q20 How do you handle Distributed Transactions?

Answer

Avoid 2PC (two-phase commit) — it's slow and couples services. Instead use the Saga pattern: break the transaction into local transactions each publishing an event. If one step fails, compensating transactions undo the previous steps.

Two flavours: Choreography (services react to events) and Orchestration (a central orchestrator drives the flow).

Example

```
// Orchestration Saga with MassTransit
public class OrderSaga : MassTransitStateMachine<OrderState>
{
    public OrderSaga()
    {
        Initially(
            When(OrderPlaced)
                .Then(ctx => ctx.Saga.OrderId = ctx.Message.OrderId)
                .TransitionTo(PaymentPending)
                .Publish(ctx => new ProcessPaymentCommand(ctx.Saga.OrderId));

            During(PaymentPending,
                When(PaymentFailed)
                    .Publish(ctx => new CancelOrderCommand(ctx.Saga.OrderId))
                    .TransitionTo(Cancelled));
    }
}
```

Q21 How do you ensure Idempotency in Event-Driven Systems?

Answer

Idempotency means processing the same message twice produces the same result. Track processed message IDs in a database or cache. Before processing, check if the ID was already handled — if yes, skip.

Example

```
public async Task HandleAsync(OrderPlacedEvent evt)
{
    // Check if already processed
    bool alreadyDone = await _db.ProcessedMessages
        .AnyAsync(m => m.MessageId == evt.MessageId);
    if (alreadyDone) return; // skip duplicate

    // Process the event
    await _db.ProcessedMessages.AddAsync(
        new ProcessedMessage { MessageId = evt.MessageId });
    await _inventory.ReserveAsync(evt.OrderId);
    await _db.SaveChangesAsync();
}
```


Q22 How do you prevent duplicate message processing?

Answer

Use an outbox table (Transactional Outbox pattern) to atomically save the event with your business data. A background poller sends it. On the consumer side, deduplicate using a unique MessageId stored in a processed-messages table.

Example

```
// Outbox pattern: save event in same DB transaction as business data
await using var tx = await _db.Database.BeginTransactionAsync();

_db.Orders.Add(new Order());
_db.OutboxMessages.Add(new OutboxMessage {
    Type = nameof(OrderPlacedEvent),
    Payload = JsonSerializer.Serialize(new OrderPlacedEvent(new Order.Id)),
    CreatedAt = DateTime.UtcNow
});

await _db.SaveChangesAsync();
await tx.CommitAsync();
// Separate worker polls OutboxMessages and publishes to broker
```

Q23 What is the Saga Pattern and where have you used it?

Answer

Saga coordinates long-running business processes across multiple services without a distributed transaction. Each service completes its local transaction and emits an event. If any step fails, compensating actions undo previous steps.

Real use case: e-commerce checkout — Reserve Inventory → Charge Payment → Send Confirmation. If Payment fails, inventory reservation is cancelled.

Example

```
// Compensating action example
public async Task HandlePaymentFailedAsync(PaymentFailedEvent evt)
{
    // Compensate: release the reserved inventory
    await _inventoryClient.ReleaseReservationAsync(evt.OrderId);

    // Notify the customer
    await _notificationClient.SendAsync(new Notification {
        UserId = evt.UserId,
        Message = "Payment failed, your order has been cancelled."
    });

    await _orderRepository.MarkCancelledAsync(evt.OrderId);
}
```

Q24 How do you implement Retry, Circuit Breaker, and Timeout patterns?

Answer

Use Polly (or Microsoft.Extensions.Http.Resilience in .NET 8+). Retry handles transient failures. Circuit Breaker stops calling a failing service to give it time to recover. Timeout prevents a slow service from blocking your threads forever.

Example

```
// .NET 8 resilience pipeline
builder.Services.AddHttpClient("payments")
    .AddResilienceHandler("default", pipeline =>
    {
        pipeline.AddRetry(new HttpRetryStrategyOptions
        {
            MaxRetryAttempts = 3,
            Delay = TimeSpan.FromSeconds(1),
            BackoffType = DelayBackoffType.Exponential
        });
    });
```

```

pipeline.AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions
{
    FailureRatio      = 0.5,
    SamplingDuration  = TimeSpan.FromSeconds(10),
    BreakDuration     = TimeSpan.FromSeconds(30)
});
pipeline.AddTimeout(TimeSpan.FromSeconds(5));
});

```

■ Messaging & Event-Driven Architecture

Q25 How have you used Azure Service Bus in your projects?

Answer

Azure Service Bus provides reliable, ordered message delivery. I've used Queues for point-to-point commands (e.g. send email) and Topics+Subscriptions for pub/sub events (e.g. order placed → inventory, billing, and notification services all receive it).

Key features: sessions for ordered processing, dead-letter queue for poison messages, message lock renewal to prevent redelivery during slow processing, and at-least-once delivery guarantee.

Example

```

// Publish to a topic
await using var client = new ServiceBusClient(connectionString);
var sender = client.CreateSender("order-events");
var message = new ServiceBusMessage(
    JsonSerializer.Serialize(new OrderPlacedEvent(orderId)))
{
    ContentType = "application/json",
    MessageId = Guid.NewGuid().ToString(),
    CorrelationId = correlationId
};
await sender.SendMessageAsync(message);

```

Q26 When would you choose Queue over Topic in Azure Service Bus?

Answer

Queue — one sender, one receiver. Use for commands or work items where exactly one service should process the message. Example: resize-image jobs.

Topic — one sender, many receivers (subscriptions). Use for events where multiple services care about the same thing. Example: OrderPlaced event consumed by billing, warehouse, and notifications independently.

Example

```

// Queue - single consumer
var receiver = client.CreateReceiver("resize-image-queue");

// Topic - multiple independent consumers
// Each subscription gets its own copy of the message:
// "orders/billing-sub", "orders/warehouse-sub", "orders/notify-sub"
var billingReceiver = client.CreateReceiver(
    topicName: "order-events",
    subscriptionName: "billing-sub");

```

Q27 How do you handle message failures and dead-letter queues?

Answer

When a message fails processing (exception or exceeds max delivery count), Service Bus moves it to the dead-letter queue (DLQ) automatically. Set up a separate worker to read the DLQ, log the details, alert on-call, and either retry with a fix or discard after investigation.

Example

```

// Processor with error handling

```

```
processor.ProcessMessageAsync += async args =>
{
    try
    {
        var evt = JsonSerializer.Deserialize<OrderEvent>(args.Message.Body);
        await HandleAsync(evt!);
        await args.CompleteMessageAsync(args.Message); // ACK
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed processing {MsgId}", args.Message.MessageId);
        // After MaxDeliveryCount retries, ASB auto-moves to DLQ
        await args.AbandonMessageAsync(args.Message); // NACK - retry
    }
};
```

Q28 How do you guarantee message delivery in distributed systems?

Answer

At-least-once delivery: broker retains the message until consumer explicitly ACKs it. If the consumer crashes before ACK, the broker redelivers. Combined with idempotent consumers, this is the practical standard.

At-most-once: fire-and-forget, simpler but can lose messages. Exactly-once: very hard, usually needs transactional producers (Kafka) + idempotent consumers.

Example

```
// Only ACK after successful processing
var message = await receiver.ReceiveMessageAsync(maxWaitTime: TimeSpan.FromSeconds(5));
if (message is null) return;

try
{
    await ProcessAsync(message);
    await receiver.CompleteMessageAsync(message); // ACK - removes from queue
}
catch
{
    await receiver.AbandonMessageAsync(message); // NACK - broker retries
}
```

Q29 How do you troubleshoot message processing delays?

Answer

Check these in order: 1. Queue depth metric — is backlog growing? (consumer too slow) 2. Consumer CPU/memory — is the pod throttled? 3. Lock duration — are messages being abandoned and re-queued? 4. Downstream dependencies — is DB or external API slow? 5. Dead-letter queue depth — lots of failures causing retries?

Fix: scale out consumers, increase lock duration, add batching.

Example

```
// Azure Monitor KQL - queue depth alert
ServiceBusMetrics
| where MetricName == "ActiveMessages"
| where EntityName == "order-events"
| where Average > 5000
| project TimeGenerated, Average
| render timechart

// Kubernetes: scale consumers based on queue depth
// KEDA ScaledObject with Azure Service Bus trigger
// minReplicaCount: 1, maxReplicaCount: 20
// queueLength: 100 (scale up when > 100 messages per replica)
```

Q30 What monitoring strategy do you use for asynchronous systems?

Answer

Correlation IDs across all services + structured logging (Serilog/NLog → Elasticsearch or Azure Monitor). Distributed tracing with OpenTelemetry — every message carries a trace context so you can see the full flow in Jaeger or Application Insights.

Metrics to watch: queue depth, processing rate, error rate, DLQ depth, and end-to-end latency.

Example

```
// Propagate trace context in messages
var activity = Activity.Current;
var message = new ServiceBusMessage(body)
{
    ApplicationProperties =
    {
        ["traceparent"] = activity?.Id,
        ["tracestate"] = activity?.TraceStateString
    }
};

// On consumer side, restore context
var parentId = args.Message.ApplicationProperties["traceparent"]?.ToString();
using var linkedActivity = activitySource.StartActivity(
    "ProcessOrder", ActivityKind.Consumer,
    parentContext: ActivityContext.Parse(parentId, null));
```

■ Cloud, DevOps & Observability

Q31 How do you containerize a .NET application using Docker?

Answer

Write a multi-stage Dockerfile: stage 1 restores and builds, stage 2 is a slim runtime image. This keeps the final image small (~100MB vs ~600MB) and excludes SDK and build tools.

Best practice: run as non-root user, use .dockerignore, pin base image tags.

Example

```
# Stage 1 - build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

# Stage 2 - runtime only (small image)
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
# Run as non-root
USER app
ENTRYPOINT ["dotnet", "MyApi.dll"]
```

Q32 What Kubernetes concepts should every backend developer know?

Answer

Pod — smallest deployable unit (one or more containers sharing network). Deployment — manages replicas and rolling updates. Service — stable DNS + load balancing across pods. ConfigMap / Secret — externalise config and sensitive values. Ingress — HTTP routing into the cluster. HPA (Horizontal Pod Autoscaler) — scale pods based on CPU/memory or custom metrics. Namespace — logical isolation.

Example

```
# deployment.yaml - rolling update, 3 replicas
apiVersion: apps/v1
```

```

kind: Deployment
metadata:
  name: order-api
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    spec:
      containers:
      - name: api
        image: myregistry/order-api:1.0.5
        resources:
          requests: { cpu: "250m", memory: "256Mi" }
          limits: { cpu: "500m", memory: "512Mi" }

```

Q33 How do you manage secrets and sensitive configurations?

Answer

Never store secrets in code or Docker images. Options: Kubernetes Secrets (base64, not encrypted at rest by default — use Sealed Secrets or external-secrets-operator), Azure Key Vault with Managed Identity, AWS Secrets Manager, or HashiCorp Vault.

In .NET: use Azure Key Vault provider with IConfiguration so secrets are injected the same as any config value.

Example

```

// Azure Key Vault in Program.cs
builder.Configuration.AddAzureKeyVault(
    new Uri($"https://{vaultName}.vault.azure.net/"),
    new DefaultAzureCredential()); // uses Managed Identity in production

// Access like normal config - no code change needed
var connStr = builder.Configuration["ConnectionStrings:Db"];

// Kubernetes: mount secret as env var
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password

```

Q34 How do you implement centralised logging across Microservices?

Answer

Every service writes structured JSON logs with a correlation/trace ID. Ship logs to a central store: Elasticsearch (ELK stack), Azure Monitor, or Datadog. Use Serilog with enrichers to add service name, environment, and trace ID automatically.

Example

```

// Serilog setup with structured JSON output
Log.Logger = new LoggerConfiguration()
    .Enrich.WithProperty("Service", "OrderService")
    .Enrich.WithProperty("Environment", env.EnvironmentName)
    .Enrich.FromLogContext() // picks up correlation ID from scope
    .WriteTo.Console(new JsonFormatter())
    .WriteTo.Elasticsearch(new ElasticsearchSinkOptions(
        new Uri(config["ElasticUri"]!)) { AutoRegisterTemplate = true })
    .CreateLogger();

// In a request handler - correlation ID auto-enriched
_logger.LogInformation("Order {OrderId} placed for {CustomerId}",
    order.Id, order.CustomerId);

```

Q35 How have you used Application Insights, OpenTelemetry, or Azure Monitor?

Answer

OpenTelemetry is the vendor-neutral standard — I instrument the app once and export to any backend (App Insights, Jaeger, Datadog). It captures traces (request flows), metrics (counters, histograms), and logs.

Application Insights provides auto-instrumentation for .NET with minimal config.

Example

```
// OpenTelemetry setup in .NET 8
builder.Services.AddOpenTelemetry()
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddEntityFrameworkCoreInstrumentation()
        .AddAzureMonitorTraceExporter(o =>
            o.ConnectionString = config["AppInsights:ConnectionString"]))
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation()
        .AddAzureMonitorMetricExporter());
```

Q36 How do you investigate production issues in cloud-hosted applications?

Answer

A structured debugging process: 1. Check alerts and dashboards — error rate spike? latency increase? 2. Search centralised logs by correlation ID of the failing requests. 3. Look at distributed traces — which service or DB call is slow? 4. Check infrastructure — pod CPU/memory, DB connection pool usage. 5. Reproduce in staging if possible, then deploy the fix. 6. Watch metrics after deploy and write a post-mortem.

Example

```
// Structured logging makes searching easy
_logger.LogError(ex,
    "Order processing failed {@OrderId} for {@CustomerId}. " +
    "CorrelationId: {CorrelationId}",
    orderId, customerId, correlationId);

// KQL in Application Insights - find all errors in last hour
exceptions
| where timestamp > ago(1h)
| where customDimensions["Service"] == "OrderService"
| summarize count() by outerMessage
| order by count_ desc
```

■ Security, API Design & Advanced Topics

Q37 How do you implement Authentication and Authorization in ASP.NET Core?

Answer

Authentication = who are you. Use JWT Bearer tokens or ASP.NET Core Identity. Authorization = what can you do. Use [Authorize] with roles or policy-based rules.

For APIs: issue a JWT on login, validate on every request via middleware. For fine-grained rules use policies.

Example

```
// Validate JWT on every request
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opts =>
    {
        opts.TokenValidationParameters = new TokenValidationParameters
        {
            ValidIssuer = config["Jwt:Issuer"],
            ValidAudience = config["Jwt:Audience"],
```

```

        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(config["Jwt:Key"]!)),
        ValidateLifetime = true
    };
});

// Policy-based authorization
builder.Services.AddAuthorization(opts =>
    opts.AddPolicy("AdminOnly", p => p.RequireRole("Admin")));

[Authorize(Policy = "AdminOnly")]
[HttpDelete("orders/{id}")]
public IActionResult DeleteOrder(int id) { ... }

```

Q38 What is CQRS and when should you use it?

Answer

CQRS = Command Query Responsibility Segregation. Separate the write model (Commands that change state) from the read model (Queries that return data). This lets you optimise each side independently — e.g., write to a normalised SQL DB, read from a denormalised read-optimised store or Redis.

Use when read/write loads differ significantly or when domain complexity justifies it.

Example

```

// Command - changes state, returns nothing (or an ID)
public record PlaceOrderCommand(Guid CustomerId, List<OrderItem> Items);

public class PlaceOrderHandler(AppDbContext db, IEventBus bus)
    : IRequestHandler<PlaceOrderCommand, Guid>
{
    public async Task<Guid> Handle(PlaceOrderCommand cmd, CancellationToken ct)
    {
        var order = new Order(cmd.CustomerId, cmd.Items);
        db.Orders.Add(order);
        await db.SaveChangesAsync(ct);
        await bus.PublishAsync(new OrderPlacedEvent(order.Id));
        return order.Id;
    }
}

// Query - read-only, can hit a replica or cache
public record GetOrderQuery(Guid OrderId);
public class GetOrderHandler(IReadDb readDb) : IRequestHandler<GetOrderQuery, OrderDto>
{
    public Task<OrderDto> Handle(GetOrderQuery q, CancellationToken ct)
        => readDb.Orders.FindAsync(q.OrderId, ct);
}

```

Q39 How does GraphQL differ from REST, and when would you choose it?

Answer

REST: fixed endpoints, fixed response shapes. Client often gets too much or too little data (over/under-fetching). GraphQL: single endpoint, client specifies exactly what fields it needs. Great for mobile apps where bandwidth matters and when many different clients consume the same API.

In .NET use Hot Chocolate or Strawberry Shake.

Example

```

// Hot Chocolate schema definition
public class Query
{
    [UseProjection, UseFiltering, UseSorting]
    public IQueryable<Order> GetOrders(AppDbContext db) => db.Orders;
}

```

```
// Client sends only what it needs
/*
query {
  orders(where: { status: { eq: "Pending" } }) {
    id
    total
    customer { name email } // no phone, no address wasted bandwidth
  }
}
*/

// Register in Program.cs
builder.Services
    .AddGraphQLServer()
    .AddQueryType<Query>()
    .AddProjections()
    .AddFiltering()
    .AddSorting();
```

Q40 Explain gRPC and when you'd use it over REST.

Answer

gRPC uses Protocol Buffers (binary format) over HTTP/2. It is faster than JSON/REST, supports bidirectional streaming, and generates strongly typed clients from a .proto contract.

Use gRPC for: internal service-to-service communication where performance matters, streaming data, and polyglot environments where the contract is shared.

Example

```
// orders.proto - shared contract
syntax = "proto3";
service OrderService {
  rpc GetOrder (GetOrderRequest) returns (OrderResponse);
  rpc StreamOrders (Empty) returns (stream OrderResponse); // server streaming
}
message GetOrderRequest { int32 id = 1; }
message OrderResponse { int32 id = 1; string status = 2; double total = 3; }

// C# server implementation
public class OrderGrpcService : OrderService.OrderServiceBase
{
    public override async Task<OrderResponse> GetOrder(
        GetOrderRequest req, ServerCallContext ctx)
    {
        {
            var order = await _repo.FindAsync(req.Id);
            return new OrderResponse { Id = order.Id, Status = order.Status,
                                     Total = (double)order.Total };
        }
    }
}
```

Q41 How do you implement Rate Limiting in ASP.NET Core?

Answer

Rate limiting prevents API abuse. .NET 7+ has built-in rate limiting middleware supporting Fixed Window, Sliding Window, Token Bucket, and Concurrency limiters. You can limit by IP, user ID, or API key.

Example

```
builder.Services.AddRateLimiter(opts =>
{
    opts.AddFixedWindowLimiter("api", cfg =>
    {
        cfg.Window          = TimeSpan.FromMinutes(1);
        cfg.PermitLimit      = 100;
```



```

        cfg.QueueLimit = 0;
        cfg.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });
    opts.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
});

app.UseRateLimiter();

[EnableRateLimiting("api")]
[HttpGet("products")]
public IActionResult GetProducts() { ... }

```

Q42 What is the difference between Optimistic and Pessimistic Concurrency?

Answer

Pessimistic: lock the row when you read it — no one else can update until you release the lock. Safe but reduces throughput. Good for high-contention, short transactions.

Optimistic: no lock on read. On update, check that the row hasn't changed (via a timestamp or row version). If it has, throw a concurrency exception and let the caller retry. Better for most web apps.

Example

```

// EF Core optimistic concurrency with row version
public class Product
{
    public int Id { get; set; }
    public decimal Price { get; set; }
    [Timestamp] // EF Core maps this to SQL rowversion
    public byte[] RowVersion { get; set; } = null!;
}

try
{
    product.Price = newPrice;
    await db.SaveChangesAsync(); // checks RowVersion automatically
}
catch (DbUpdateConcurrencyException ex)
{
    // Another user updated the row first
    var entry = ex.Entries.Single();
    var dbValues = await entry.GetDatabaseValuesAsync();
    // Merge or inform user of conflict
}

```

Q43 How do you implement Health Checks in ASP.NET Core?

Answer

Health checks tell Kubernetes or a load balancer whether your service is ready to receive traffic. Expose /health/live (is the process alive?) and /health/ready (are dependencies — DB, cache — reachable?).

Example

```

builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>("database")
    .AddRedis(config["Redis:ConnectionString"]!, "redis")
    .AddUrlGroup(new Uri("https://payment-api/health"), "payment-api");

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = _ => false // only checks process is alive
});
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});

```

```
# kubernetes deployment.yaml probes
livenessProbe:
  httpGet: { path: /health/live, port: 8080 }
  initialDelaySeconds: 5
readinessProbe:
  httpGet: { path: /health/ready, port: 8080 }
  initialDelaySeconds: 10
```

Q44 What is the Outbox Pattern and why is it important?

Answer

The Outbox Pattern solves the dual-write problem: you need to save data to your DB AND publish an event atomically. Without the outbox, a crash between the two leaves data inconsistent.

Solution: write the event to an outbox table in the same DB transaction as your business data. A separate poller reads the outbox and publishes to the broker.

Example

```
// Atomic: save order + event in one transaction
await using var tx = await db.Database.BeginTransactionAsync();

db.Orders.Add(order);
db.OutboxMessages.Add(new OutboxMessage {
    Id = Guid.NewGuid(),
    Type = "OrderPlaced",
    Payload = JsonSerializer.Serialize(new OrderPlacedEvent(order.Id)),
    CreatedAt = DateTime.UtcNow,
    Sent = false
});
await db.SaveChangesAsync();
await tx.CommitAsync();

// Background worker (separate hosted service)
// SELECT * FROM OutboxMessages WHERE Sent = false
// Publish to broker → UPDATE Sent = true
```

Q45 How do you implement API Versioning in ASP.NET Core?

Answer

API versioning lets you evolve your API without breaking existing clients. Common strategies: URL path (/api/v1/orders), query string (?api-version=1.0), or HTTP header (Api-Version: 1.0). Use `Asp.Versioning.Http` NuGet package.

Example

```
builder.Services.AddApiVersioning(opts =>
{
    opts.DefaultApiVersion = new ApiVersion(1, 0);
    opts.AssumeDefaultVersionWhenUnspecified = true;
    opts.ReportApiVersions = true; // adds api-supported-versions header
    opts.ApiVersionReader = ApiVersionReader.Combine(
        new UrlSegmentApiVersionReader(),
        new HeaderApiVersionReader("Api-Version"));
});

[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/orders")]
public class OrdersV1Controller : ControllerBase { ... }

[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/orders")]
public class OrdersV2Controller : ControllerBase
{
    // New response shape, extra fields, different behaviour
}
```

```
}
```

Q46 How do you handle Background Jobs and Scheduled Tasks in .NET?

Answer

Options: IHostedService / BackgroundService for simple recurring tasks in-process, Hangfire for persistent scheduled/recurring jobs with a dashboard, and Azure Functions / AWS Lambda for event-triggered serverless jobs.

Use Hangfire when you need job persistence, retries, and visibility without a full message broker.

Example

```
// BackgroundService - runs in the same process
public class OrderCleanupService(IServiceScopeFactory factory)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            await Task.Delay(TimeSpan.FromHours(1), ct);
            using var scope = factory.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
            var stale = await db.Orders
                .Where(o => o.Status == "Abandoned"
                    && o.CreatedAt < DateTime.UtcNow.AddDays(-7))
                .ExecuteDeleteAsync(ct);
            _logger.LogInformation("Cleaned {Count} stale orders", stale);
        }
    }
}
```

Q47 How do you design a CI/CD pipeline for a .NET microservices application?

Answer

Typical pipeline: Source (Git push) → Build & Test → Docker Build & Push → Deploy to staging → Integration Tests → Deploy to production with blue/green or canary strategy.

Tools: GitHub Actions / Azure DevOps for CI, Helm + kubectl or Argo CD for Kubernetes deployments.

Example

```
# GitHub Actions - .NET build, test, Docker push
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '8.x' }
      - run: dotnet restore
      - run: dotnet test --no-restore --collect:"XPlat Code Coverage"
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: myregistry/order-api:${{ github.sha }}
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: |
          helm upgrade order-api ./charts/order-api \
            --set image.tag=${{ github.sha }} \
            --namespace production --wait
```

Q48

What are the SOLID principles and how do you apply them in .NET?

Answer

S — Single Responsibility: one class does one job. If it handles DB access AND sends emails, split it into two classes. O — Open/Closed: add new behaviour by extending (new class/strategy), not by editing existing working code. L — Liskov Substitution: a subclass must be safely usable wherever the parent is used — no surprises for the caller. I — Interface Segregation: prefer many small focused interfaces over one large interface that forces classes to implement methods they don't need. D — Dependency Inversion: depend on abstractions (interfaces), not concrete classes. This is what makes constructor injection and unit testing with mocks possible.

Example

```
// Violates SRP - does too many things
public class OrderService
{
    public void PlaceOrder(Order o) { SaveToDb(o); SendEmail(o); LogAudit(o); }
}

// Follows SRP + DIP - each concern is separate and injected
public class OrderService(IOrderRepository repo,
                        IEmailService email,
                        IAuditLogger audit)
{
    public async Task PlaceOrderAsync(Order o)
    {
        await repo.SaveAsync(o);
        await email.SendConfirmationAsync(o);
        await audit.LogAsync($"Order {o.Id} placed");
    }
}
```

Q49

How do you write Unit Tests and Integration Tests for .NET APIs?

Answer

Unit tests: test a single class in isolation using mocks (Moq, NSubstitute). Fast, no DB or network. Integration tests: test the full HTTP pipeline using WebApplicationFactory, a real (or in-memory/testcontainers) DB. Slower but catch wiring bugs.

Example

```
// Unit test - mock the dependency
public class OrderServiceTests
{
    [Fact]
    public async Task PlaceOrder_ShouldSaveAndPublish()
    {
        var repoMock = Substitute.For<IOrderRepository>();
        var busMock = Substitute.For<IEventBus>();
        var sut = new OrderService(repoMock, busMock);

        await sut.PlaceOrderAsync(new Order { CustomerId = Guid.NewGuid() });

        await repoMock.Received(1).SaveAsync(Arg.Any<Order>());
        await busMock.Received(1).PublishAsync(Arg.Any<OrderPlacedEvent>());
    }
}

// Integration test - real HTTP pipeline
public class OrdersApiTests(WebApplicationFactory<Program> factory)
    : IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task PostOrder_Returns201()
    {
        var client = factory.CreateClient();
        var response = await client.PostAsJsonAsync("/api/orders",
            new { CustomerId = Guid.NewGuid() });
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    }
}
```

```
}  
}
```

Q50 How do you implement Event Sourcing?

Answer

Event Sourcing stores every state change as an immutable event rather than overwriting the current state. To rebuild current state you replay events. Audit trail is free, and you can time-travel to any past state.

Combine with CQRS: write side appends events, read side projects events into read models.

Example

```
// Events are facts - immutable records  
public record OrderPlaced(Guid OrderId, Guid CustomerId, decimal Total, DateTime At);  
public record PaymentReceived(Guid OrderId, decimal Amount, DateTime At);  
public record OrderShipped(Guid OrderId, string TrackingNumber, DateTime At);  
  
// Aggregate rebuilds itself by replaying events  
public class Order  
{  
    public Guid Id { get; private set; }  
    public string Status { get; private set; } = "New";  
  
    public static Order Replay(IEnumerable<object> events)  
    {  
        var order = new Order();  
        foreach (var e in events) order.Apply(e);  
        return order;  
    }  
  
    private void Apply(object evt)  
    {  
        switch (evt)  
        {  
            case OrderPlaced p:    Id = p.OrderId; Status = "Placed"; break;  
            case PaymentReceived: Status = "Paid";      break;  
            case OrderShipped:     Status = "Shipped";   break;  
        }  
    }  
}
```